

The Firewall Ingestor

Reading the security guard's logbook

What this chapter is about

A perimeter firewall is the most knowledgeable single device in any organisation's network. Every branch office is connected through it, every public-facing service is mapped through it, every "trust this zone, distrust that zone" rule lives in it. If we can politely *copy* what the firewall already knows, we get a complete map of the company's network in one shot — without scanning anything ourselves yet.

This chapter builds the **Firewall Ingestor**: a piece of code that logs into the firewall (read-only), pulls a fixed set of tables, and writes them into our database as observations.

Real-life analogy. *Imagine a 30-storey office building with a security desk at the front gate. The guard has five register books on the desk: (1) which lifts go to which floors, (2) where every internal phone wire runs, (3) a logbook of every visitor who came in this morning, (4) the locked-tunnel map to other branch offices, and (5) the rule book of who is allowed where. We send a polite intern down to the desk, say "please can I photocopy these registers?", and walk away with a near-complete picture of the whole building. The guard never gets up; nothing in the building is disturbed. That intern is the Firewall Ingestor.*

Words you'll see (and what they mean)

Firewall. A device that sits between your internal network and the outside world (or between two zones), inspects every packet, and either lets it pass or blocks it. The Indian-English equivalent is "the network's gatekeeper."

Perimeter firewall. The firewall that sits at the very edge — between your office and the public Internet. The most senior gatekeeper in the building.

Interface. A physical or logical port on the firewall. `port1` , `ethernet1/3` , `vlan100` . Each has its own IP address and is connected to a different segment of the network.

Route / Routing table. A list of "if a packet wants to reach this network, send it to this neighbour next." Like a courier who knows "anything for South Mumbai goes to dispatch hub #7."

ARP (Address Resolution Protocol). The mechanism that maps an IP address to a MAC address on the same local network. The ARP table is "I just spoke to 192.168.1.50 and its hardware address is aa:bb:cc:11:22:33." Useful because it tells us a host actually exists right now.

MAC (Media Access Control) address. The hardware-burned-in address of a network card, like `aa:bb:cc:dd:ee:ff` . Globally unique. An IP can change; a MAC usually doesn't.

NAT (Network Address Translation). Rewriting addresses on the fly. Two flavours: **SNAT** (Source NAT — "many internal users appear to the Internet as one public IP") and **DNAT** (Destination NAT, also called a VIP — "anyone hitting our public IP on port 443 actually gets routed to internal server 10.0.5.20"). The DNAT table tells us which public services map to which internal hosts.

VIP (Virtual IP). Vendor name for a published service: a public-facing IP+port that maps to an internal host+port. Same idea as DNAT.

Zone. A named group of interfaces — "trust", "untrust", "dmz", "guest-wifi". Policy rules are written between zones: "allow from trust to dmz on port 443."

VPN (Virtual Private Network) tunnel. An encrypted pipe between two firewalls (typically HQ ↔ branch). The packets travel over the public Internet but inside the pipe they look like a single private network. A site-to-site IPsec tunnel binds two specific subnets together — the "proxy IDs" — and that binding is the most reliable way to learn that "branch Pune's 10.20.0.0/16 is reachable through this tunnel from HQ."

BGP (Border Gateway Protocol) and OSPF (Open Shortest Path First). Two protocols that let routers tell each other "I can reach network X." If your firewall speaks BGP/OSPF, its neighbour table is a free list of every other router in the company that's been advertised to it.

API (Application Programming Interface). The doorway we use to talk to the firewall programmatically — REST, XML, or SSH-CLI depending on vendor.

REST (REpresentational State Transfer) API. The modern style — HTTP requests, JSON responses. Easy.

XML (eXtensible Markup Language) API. The older style. Same idea but with XML responses. Palo Alto still uses this for live status data.

CLI (Command Line Interface). The text terminal you SSH into and type `show route` on. Slowest and most fragile, but sometimes the only option.

SSH (Secure SHell). Encrypted remote-terminal protocol. We log in over SSH, run commands, parse the output text.

vsys / VDOM / context / VSX. Vendor names for "many virtual firewalls inside one physical firewall." Palo Alto = vsys, Fortinet = VDOM, Cisco ASA = context, Check Point = VSX virtual system. If a customer has more than one of these, every API call must say which one you mean — otherwise you'll see only a slice of their network and think that's the whole picture.

RBAC (Role-Based Access Control) read-only role. A user account that can only read, never change anything. We *always* log into customer firewalls with this kind of account. If we make a bug, the worst we can do is read too much; we can never accidentally break their firewall config.

What we're building (the picture)

Customer's office

Our SaaS cloud

Perimeter firewall
PAN-OS / FortiGate
ASA / Check Point

← HTTPS / SSH (read-only) —

Collector
(Go agent)

pulls these tables:
— interfaces
— routes

- ARP
- NAT (DNAT/VIP)
- zones + policies
- VPN tunnels (proxy-IDs)
- BGP/OSPF neighbours

WebSocket



NestJS API
Postgres

Stored as one
DiscoverySession
+ many
Observations

What we get from the firewall (and what we don't)

What the firewall can tell us	What it cannot
Every subnet directly attached to it (HQ floors, DMZ, guest wifi)	Internal VLANs that exist behind a downstream switch
Every remote subnet reachable via routing (branch offices)	How those remote subnets are wired internally
Public-facing services (DNAT/VIP table)	Hostnames, OS, software running on each host
Branch-to-HQ encrypted tunnels and the subnets they bind	Wireless clients, VMs sharing a hypervisor, IoT devices
Hosts that have spoken through the firewall recently (ARP)	Hosts that have never ARPed through the firewall
Inter-zone policy graph: "guest can reach Internet but not HR"	Whether the actual host on 10.0.5.42 is a printer or a server

So what does Chapter 1 alone get us? A complete *subnet* and *site* map. A list of branches and how they connect. The set of public services. A partial host list. We will close the host gap with SNMP (Chapter 2) and active probing (Chapter 3).

The four vendors, in plain English

We are going to support four vendors because they cover ~90% of enterprise customers. We build them **one at a time** — Fortinet first because it's the friendliest, Cisco ASA last because it's the most painful.

1. FortiGate (FortiOS 7.2 and newer) — start here

Modern REST API. JSON in, JSON out. Two endpoint families:

- `/api/v2/cmd/*` — the **configuration** (firewall.policy, system.zone, vpn.ipsec.phase1, etc.).
- `/api/v2/monitor/*` — the **live state** (router/ipv4 = current routing table, system/arp, vpn/ipsec, router/bgp/neighbors, system/interface).

Authentication is a bearer token from a "REST-API admin" user (`config system api-user`) limited to a read-only profile and locked to your collector's source IP (`trusthost`). One thing to remember: every URL must include `?vdom=...`, even if the customer only has one VDOM.

2. Palo Alto (PAN-OS 10.1 and newer)

Two APIs awkwardly stitched together:

- **REST API** at `/restapi/v10.x/` — for **config** only (zones, policies, NAT rules, IPSec configuration). Clean JSON.
- **XML API** at `/api/?type=op&cmd=...` — for **live state**. We send a tiny XML command like `<show><routing><route/></routing></show>` and get back a giant XML blob.

Authentication is an API key obtained by POSTing to `/api/?type=keygen`. The key is bound to the admin password — if password rotates, the key silently dies.

3. Check Point (Gaia R81 and newer)

Three surfaces for one vendor (yes, really):

- The **gateway** itself exposes Gaia REST (`/gaia_api/v1.x/`) — interfaces, ARP, static routes.
- The **Security Management Server (SMS)** exposes the Management API (`mgmt_cli` over REST) — policies, NAT, VPN community objects, network objects.
- BGP/OSPF state and live IPSec tunnel state are not first-class in any REST endpoint — we fall back to **SSH clish** and run commands like `show bgp peers`, `vpn tu tlist`.

If the customer uses VSX (multiple virtual systems on one physical box), every call needs a `vsid`; without it you only see VSO and miss everything else.

4. Cisco ASA / Firepower (FTD) — keep for last

The painful one. Cisco's ASA REST API was deprecated and removed; the newer FTD has FDM/FMC REST APIs but they only cover *config*, never live route/ARP/session tables. So our primary path is **SSH scraping**:

```
ssh admin@asa-edge> show route
> show arp
> show nat detail
> show crypto ipsec sa
> show bgp summary
> show ospf neighbor
```

We use a Go library called **scrapligo** to handle the SSH session and the prompt/pager dance, and a parser called **gotextfsm** with the open-source **ntc-templates** library to turn the text output into structured data. This is real work — plan two full sprints for ASA alone.

Multi-context ASA: every command must run inside `changeto context <name>` first. Enumerate contexts, then loop.

Step-by-step plan

Step 1 — Build the vendor-neutral interface

Define a Go interface in `internal/discovery/firewall/firewall.go` :

```
type Ingestor interface {
    Login(ctx context.Context, creds Creds) error
    ListContexts(ctx context.Context) ([]Context, error)
    GetInterfaces(ctx context.Context, c Context) ([]Interface, error)
    GetRoutes(ctx context.Context, c Context) ([]Route, error)
    GetArpTable(ctx context.Context, c Context) ([]ArpEntry, error)
    GetNatRules(ctx context.Context, c Context) ([]NatRule, error)
    GetZones(ctx context.Context, c Context) ([]Zone, error)
    GetPolicies(ctx context.Context, c Context) ([]Policy, error)
    GetVpnTunnels(ctx context.Context, c Context) ([]VpnTunnel, error)
    GetBgpNeighbors(ctx context.Context, c Context) ([]BgpNeighbor, error)
    GetOspfNeighbors(ctx context.Context, c Context) ([]OspfNeighbor, error)
    Logout() error
}
```

Every vendor implementation satisfies this interface. The collector loop never knows which vendor it's talking to — it just calls `GetRoutes` .

Step 2 — Implement FortiGate first

1. Use plain `net/http` , not Fortinet's SDK (their SDK is glued to Terraform and clumsy for read-only ingestion).
2. Define one Go struct per JSON response shape.

3. Always paginate with `start=0&count=1000` and loop until response shorter than count.
4. Discover VDOMs first (`/api/v2/cmdb/system/vdom`), then run every other call once per VDOM.

Step 3 — Implement Palo Alto next

1. For config endpoints: `github.com/PaloAltoNetworks/pango` is the official, generated SDK. Use it.
2. For op-state (route, ARP, tunnel, BGP, OSPF): build small `net/http` + `encoding/xml` calls. The library doesn't help here.
3. Discover virtual systems (`show system info` → multi-vsyst flag, then `show vsys`) and scope every op-cmd with `&target-vsyst=vsys2` or run inside the vsyst context.
4. Group route entries by their `<virtual-router>` element — one PAN can have many VRs.

Step 4 — Implement Check Point

1. Login to Gaia REST: `POST /gaia_api/v1.5/login` → returns a `sid`. Use it as `x-chkp-sid` header.
2. Login to the Management API: `POST /web_api/login` → returns a session id.
3. For the gaps (`vpn tu` , BGP peers), fall back to SSH clish via **scraplgo**.
4. If VSX, enumerate virtual systems and pass `vsid` on every call.

Step 5 — Implement Cisco ASA last

1. Connect with **scraplgo** using the `cisco_asa` platform — it knows the prompt, the `--More--` pager, and the `terminal pager 0` trick.
2. Enumerate contexts with `show context` ; loop with `changeto context <name>` .
3. For each context, run our six show commands.
4. Pipe the text through **gotextfsm** with the matching **ntc-templates** template (e.g. `cisco_asa_show_route.textfsm`) — get back structured records.

Step 6 — Stream observations back

For each table the ingestor returns, the collector wraps the rows in `ScanJobChunk` frames and sends them. Each row becomes one `observation` in Postgres, with `source_collector_id` , `session_id` , and `seen_at` . The post-processing service then upserts into `site` , `subnet` , `nat_rule` , `vpn_tunnel` , etc.

How we test Chapter 1

Lab setup. Spin up free / eval images of all four vendors:

- FortiGate VM eval (free 15-day evaluation license)
- Palo Alto PA-VM (eval available via partner channels)
- Cisco ASAv image (or use a real ASA in your lab)
- Check Point CloudGuard Network IaaS evaluation

Configure each with a small known topology: 2 sites, 1 IPSec tunnel between them, 2 internal subnets, 1 DNAT VIP.

Test 1 — Identify subnets.

1. Trigger an ingest job against vendor X.
2. Confirm the produced subnet count matches what you configured. Off-by-one usually means you missed a sub-interface or a tunnel-bound network.

Test 2 — Identify the VPN edge.

1. Confirm the IPSec tunnel appears in the ingestor output with both sites' proxy-IDs.
2. Confirm a `NeighborEdge` row of type `ipsec_tunnel` exists between Site-A and Site-B.

Test 3 — DNAT/VIP discovery.

1. Configure one VIP that maps `203.0.113.10:443` → `10.0.5.20:443` .
2. Confirm `nat_rule` table has the right mapping with `direction=dst` .

Test 4 — Multi-context.

1. For Fortinet: configure 2 VDOMs, run ingest, confirm rows from both appear with `context` column populated.
2. Repeat for Palo Alto vsys, ASA contexts, Check Point VSX.

Test 5 — Read-only proof.

1. Snapshot the firewall config before ingest.
2. Run ingest 100 times in a loop.
3. Snapshot config after; the diff must be empty.

Things that can go wrong (and what to watch for)

Multi-context blindness. You forget to enumerate vsys/VDOM/context, and your ingestor only ever sees the default. The customer says "where's our DMZ?" and you have no answer. Always list contexts first; never assume one.

Pulling the session table by accident. Some vendors have a `session` or `conntrack` table with millions of rows. It looks tempting ("look, every active connection!") but it's runtime noise, it changes every second, and it will OOM your collector. Stick to ARP for "who is alive."

Self-signed certificates. Customers' firewalls almost always have self-signed TLS certs. Don't globally disable certificate verification — instead, store a per-tenant pinned certificate fingerprint in the credential vault, and only trust matches.

Ratecaps undocumented but real. PAN-OS XML API runs on the management plane; if you parallelise more than ~4 calls, the firewall's web GUI for the customer's admin will stutter. FMC limits to 120 req/min/user. Build a per-vendor concurrency cap.

Treating "read-only" as truly read-only. Some PAN op-commands trigger a commit-lock; some FortiOS read-only profiles still block `monitor/router/ipv4` on older builds. Always test with the exact role you'll ship, not with the global admin user.

Routes mis-keyed across VRs. A multi-VR Palo Alto returns one big route blob; if you don't group by `<virtual-router>`, you'll have routes "from" the wrong router and incorrect topology edges.

API key invalidation on password rotation. PAN-OS API keys are derived from the admin password. When the customer rotates the password (every 90 days under most security policies), your key dies silently and you stop ingesting. Detect 401s and surface an alert; don't retry forever.

Exit checklist (you can move to Chapter 2 only when ALL are true)

- ☐ All four vendors implement the `firewall.ingestor` interface (or three, and ASA is on a backlog ticket with a clear plan).

- ☐ Lab ingestion produces a subnet count matching ground truth for each vendor.
- ☐ Multi-context customers are correctly fanned out (every vsys/VDOM/context/VSX appears).
- ☐ VPN tunnel proxy-IDs become `NeighborEdge` rows linking the right Sites.
- ☐ DNAT/VIP rules are stored with internal+external IP+port and direction.
- ☐ A 100-iteration ingest loop produces a *zero-diff* on the firewall's running-config.
- ☐ Self-signed certs handled per-tenant; no global skip-verify in production code.
- ☐ A 401 from the firewall (e.g. password rotated) raises an actionable alert, not a silent retry storm.

Recap card — Chapter 1 in 5 sentences

1. The perimeter firewall already knows the company's subnets, branches, and rules — we politely copy that, we don't recreate it.
2. Build a vendor-neutral Go interface; every vendor implementation behind it speaks the firewall's native API.
3. Order: FortiGate (REST, easy) → Palo Alto (REST + XML) → Check Point (Gaia + SMS + clish) → Cisco ASA (SSH scraping with scraplgo + ntc-templates).
4. Always enumerate vsys/VDOM/context/VSX first; everything else fans out per-context.
5. Firewall ingestor gives us a complete subnet map but only a partial host map — Chapters 2 and 3 fill that gap.